CTCPU parts

James Augustus

Abstract

I started this with colour: three light channels make eight colours, and eight colours make one octal digit. The story is in Cell. I built CTCPU around a simple choice. For addition, I keep the number's normal octal digits. If the number's structure will help with multiplication, division or roots, I prepare its shadow, that is, its exponents for the small primes. This folder explains the processor's parts, starting with the smallest. Each part has its own folder, its own description and drawings. I continued this work under the name Hunter. I write about the current design, the open work and the parts from earlier designs separately.

Contents

Section	Title
1	Reading order
2	Current layout
3	All parts and their status
4	Parts from earlier designs
5	Format of the descriptions

1. Reading order

Order	Part	Folder	What it explains
1	Cell	Cell	three bits, one octal digit, eight colours
2	Register	Register	8 rows × 8 cells, 192 bits, the roles of the rows
3	Resolver	Resolver	working and tested forms of Resolver
		Gear model	Resolver explained frame by frame with dials and gears
		Resolver experiments	forms of Resolver not implemented in code
4	ALU	ALU	two layers: shadow and positional
	Adder	ALU/Adder	alignment, carry chain
	Multiplication and division	ALU/Multiply_Divide	exponent arithmetic, fraction, rough product
	Square root	ALU/Square_Root	halving exponents, symbolic root, continued fraction
5	Types	Types	eight type codes and statuses
6	Core	Core	bringing the parts together, the path of one instruction

Instruction formats are in ISA.

2. Current layout

I use one core, one thread and eight registers: $R0 \ldots R7$. Each register has eight rows of eight cells. One cell is three bits. One row is 24 bits, one register is 192 bits, and eight registers together are 1536 bits. These 192 bits also contain other representations of the number and information about it. The register does not hold a 192-bit integer. The number itself is in the first row, in eight digits. Whether the shadow rows hold the octal digits of one exponent or the levels of a ladder, and how these fit into the combined record, remains open. The shadow system can both keep a number as a binomial such as $2^k \pm r$ and factor it. This is a feature of one of the processors I am designing, and it can also do base arithmetic. FPGA experiments will be done on the forms of Resolver.

3. All parts and their status

Part	What it does	Status
Cell (xbit)	one octal digit from 0..7 in three bits	current
Register $R0..R7$	the number's complete record, 192 bits	current
rm	the number's eight digits	current
$rs1..rs4$	shadow: octal exponent digits in the counter form, levels and cut digits in the ladder form	combined record open, code for the layered ladder is open
rc	rough cache	current, exact role in reading back is open
$rpow$	scale declaration	current, cell layout is open
$rtyp$	type and statuses	type is in cell 0, the other cells are open
Resolver	tests seven primes, counts exponents, takes the residual down level by level	first stage and gear model tested, layered descent absent from the code
Adder	addition, subtraction, comparison	current
Multiplication and division	exponent addition and subtraction, fraction pair	current, rough product on the plain path
Square root	halving exponents, symbolic root, digit-by-digit root	current
Control	program counter, instruction decoder, clock	roles defined, timing open
ISA	operation codes, mode bits	Hunter draft, has open parts

4. Parts from earlier designs

The parts below were present in earlier designs. Their places in the current 192-bit layout have not yet been decided, or they are being considered within another field.

Part	In the earlier design	In the current 192-bit layout
Status field (FLAG)	eight status bits in a separate field	statuses are being considered within $rtyp$, cell layout is open
VLINK	three-bit link field in the 137-bit register	place not decided
137-bit register	earlier record layout	the 192-bit layout with 8×8 cells is used
Separate controller (PCCPU)	separate unit distributing instructions	place in the single-core design not decided
Overflow chain	carrying to neighbouring registers	connection and endpoint open
Cube, leader and workers	many cores working together	current design has one core, multiple cores are open
Simplification sieve	finding opportunities to simplify, such as matching factors and zero	defined in the July design, current layout open
Opening modes, fraction and quarantine pair	symbolic expansion and exceptions	defined in the July design, current layout open
Teaching model (NAND, adder, IAR, IR, ACC, BUS, RAM)	model that makes instruction execution visible	remains a separate teaching model

5. Format of the descriptions

Each part follows the same layout: abstract, definitions, mechanism, examples calculated by a program, recorded and rerun measurements, limits and open questions. The drawings are black and white, and every value in the examples is calculated by the program that produces the drawing. Measurements from earlier work are marked as “recorded measurement”, and rerun tests are given with their dates.

James Augustus, 2026. Licensed under CC BY 4.0. https://github.com/JamesAugustus/ctcpu-hunter